LAB 2 TASK
CampusConnect – REST API Integration

Continuation of Lab 1 • JSONPlaceholder • Postman • JavaScript fetch()

Important: Do not create or deploy your own API for this lab. Use the existing JSONPlaceholder REST API and connect it to the CampusConnect webpage created in Lab 1.

1. Objective

In Lab 1, you created the CampusConnect dashboard using HTML, CSS and JavaScript with mock/static data. In Lab 2, keep the same interface but replace the data in three existing sections with data fetched from REST APIs.

You will test the APIs using Postman, use JavaScript fetch() to retrieve the data, convert the response to JSON, and display the data dynamically in the existing UI.

2. Sections to Integrate

You must integrate these THREE CampusConnect sections that were already part of your Lab 1 project:

CampusConnect Section	API Resource	GET Endpoint	API Link	Suggested Data
Student Profile	Users	GET /users/1	https://jsonplaceholder.typicode.com/users/1	name, username, email, phone
Announcements	Posts	GET /posts?_limit=5	https://jsonplaceholder.typicode.com/posts?_limit=5	title, body
Assignments	Todos	GET /todos?userId=1&_limit=5	https://jsonplaceholder.typicode.com/todos?userId=1&_limit=5	title, completed

JSONPlaceholder is a practice REST API. Its data is not real university data; it is used only to learn API integration.

3. Step-by-Step Procedure

Step 1 — Open your Lab 1 project: Open the CampusConnect project created in Lab 1. Keep the same HTML, CSS and JavaScript structure.

Step 2 — Identify the three sections: Locate Student Profile, Announcements and Assignments in your existing webpage.

Step 3 — Open the API links: Open each API link from Section 2 in a browser or use the URL in Postman. Observe the JSON response.

Step 4 — Test Student Profile: In Postman, create a GET request for https://jsonplaceholder.typicode.com/users/1 and click Send. Check the status code and JSON response.

Step 5 — Test Announcements: In Postman, create a GET request for https://jsonplaceholder.typicode.com/posts?_limit=5 and inspect the returned posts.

Step 6 — Test Assignments: In Postman, create a GET request for https://jsonplaceholder.typicode.com/todos?userId=1&_limit=5 and inspect the returned todo items.

Step 7 — Identify fields: Decide which JSON fields will be displayed in each section. Use the Suggested Data column as a starting point.

Step 8 — Use fetch(): In JavaScript, use fetch() to call each API. You may use async/await or Promises.

Step 9 — Convert to JSON: Use response.json() to convert the API response into JavaScript data.

Step 10 — Display dynamically: Use JavaScript to update the existing CampusConnect HTML elements. Do not manually copy the API response into the HTML.

Step 11 — Add one interaction: Add one simple interaction: filter completed/pending assignments, search announcements, or refresh the API data.

Step 12 — Handle errors: Check whether the request was successful. If it fails, show a simple message such as 'Unable to load data. Please try again.'

Step 13 — Test the webpage: Open CampusConnect and confirm all three sections display data obtained from the APIs.

Step 14 — Update README.md: Document the three APIs, endpoints, HTTP method and JSON fields used.

4. API Integration Flow

CampusConnect	JavaScript	HTTP	API	JSON	UI
Student Profile	fetch()	GET	/users/1	name, email, ...	Profile card
Announcements	fetch()	GET	/posts	title, body	Announcement cards
Assignments	fetch()	GET	/todos	title, completed	Assignment list

5. Example JavaScript Approach

Example for Announcements. Use the same approach for Student Profile and Assignments.

```
async function loadAnnouncements() {
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/posts?_limit=5"
        );

        if (!response.ok) {
            throw new Error("API request failed");
        }

        const data = await response.json();
        displayAnnouncements(data);
    }
    catch (error) {
        console.error(error);
        // Show an error message in the UI
    }
}
```

The sequence is: fetch endpoint → check response → response.json() → use returned data to update the webpage.

6. One Simple Interaction

Add ONE of the following:

- Announcements: search/filter announcements by title.
- Assignments: filter Completed and Pending assignments.
- Any section: add a Refresh button that calls the API again.

7. Submission Requirements

- Updated CampusConnect project from Lab 1.
- Student Profile connected to JSONPlaceholder Users API.
- Announcements connected to JSONPlaceholder Posts API.
- Assignments connected to JSONPlaceholder Todos API.
- Postman screenshots showing the three GET requests and responses.
- Basic error handling.
- Updated README.md containing the API mapping.
- One screenshot of the final API-powered CampusConnect webpage.
- Completed API-design exercise.